

Formal Methods and Specification (SS 2025/26)

Lecture 6:

Functions, Procedures

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Complexity Reduction

software systems

...

functions, procedures

control structures

basic paths:

- assume** $x \geq 5$
- $y \leftarrow x$
- $z \leftarrow x + y$
- @ $z \geq 0$

logic, proofs

Function Specification

For using a function, it suffices to know its **interface**:

function sort(a, n):

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

function count(a, n, x)

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: $|\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

We hide the implementation (information hiding),
it may change arbitrarily (as long as it fulfills the specification)

viz C/C++: header files

Usage

function $\text{sort}(a, n)$:

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

function $\text{count}(a, n, x)$

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: $|\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

$s \leftarrow \text{sort}(a, n)$

$p \leftarrow \text{count}(s, n, v)$

@ $p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$

Correctness

Basic path:

$s \leftarrow \text{sort}(a, n)$

$p \leftarrow \text{count}(s, n, v)$

correct?

@ $p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$

Verification condition:

$[s = \text{sort}(a, n) \wedge p = \text{count}(s, n, v)] \Rightarrow$

true?

$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$

Missing: knowledge about function calls

What do we know about the used functions? input/output specifications

logical formulas?

Representation of Specification using Output Variable

function sort(a, n):

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)]$$

function count(a, n, x)

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: $|\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

The output does not have a name!?

function count(a, n, x)

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: r s.t. $r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} .$$

$$[\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$$

Representation of Specification using Function Name

function sort(a, n):

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

$$\forall a \in \mathcal{A}, n \in \mathcal{N} . \text{sorted}(\text{sort}(a, n), n) \wedge \text{perm}(a, \text{sort}(a, n), n)$$

function count(a, n, x)

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: $|\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, x \in \mathcal{N} . \text{sorted}(a, n) \Rightarrow$$

$$\text{count}(a, n, x) = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$$

Basic Path with Assumptions using Output Variables

assume $\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)]$
assume $\forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} . [\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow r = |\{i \mid i \in$
 $s \leftarrow \text{sort}(a, n)$
 $p \leftarrow \text{count}(s, n, v)$
 $@ p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|$

resulting verification condition:

additional assumptions on left-hand side of implication

Proof of Verification Condition using Output Variables

To prove the verification condition we assume the items on the left-hand side

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, b \in \mathcal{A} . b = \text{sort}(a, n) \Rightarrow [\text{sorted}(b, n) \wedge \text{perm}(a, b, n)]$$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, x, r \in \mathcal{N} .$$

$$[\text{sorted}(a, n) \wedge r = \text{count}(a, n, x)] \Rightarrow r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$$

$$s = \text{sort}(a, n)$$

$$p = \text{count}(s, n, v)$$

and prove the right-hand side

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

We start by proving properties of the result of the first program line s . From the assumption describing the function sort , we know $\text{sorted}(s, n)$ and $\text{perm}(a, s, n)$.

From the assumption describing the function count , after the choices $a \leftarrow s$, $n \leftarrow n$, $x \leftarrow v$ we know:

$$[\text{sorted}(s, n) \wedge p = \text{count}(s, n, v)] \Rightarrow p = |\{i \mid i \in \{1, \dots, n\}, s[i] < v\}|.$$

Since we know both assumptions on the left-hand side, we also know the right-hand side. Since $\text{perm}(a, s, n)$, both a and s contain exactly the same elements, and hence

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|,$$

which is what we wanted to prove.

Proof of Verification Condition using Function Names

To prove the verification condition we assume the left-hand side

$$\forall a \in \mathcal{A}, n \in \mathcal{N} . \text{sorted}(\text{sort}(a, n), n) \wedge \text{perm}(a, \text{sort}(a, n), n)$$

$$\forall a \in \mathcal{A}, n \in \mathcal{N}, x . \text{sorted}(a, n) \Rightarrow \text{count}(a, n, x) = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$$

$$s = \text{sort}(a, n)$$

$$p = \text{count}(s, n, v)$$

and prove the right-hand side

$$p = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

Due to substitutions resulting from the assumptions, it suffices to prove

$$\text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

We start by proving properties of the result of the first program line $\text{sort}(a, n)$. From the first assumption above, we know $\text{sorted}(\text{sort}(a, n), n)$ and $\text{perm}(a, \text{sort}(a, n), n)$.

From the second assumption we know, after the choices $a \leftarrow \text{sort}(a, n)$, $n \leftarrow n$, $x \leftarrow v$:

$$\text{sorted}(\text{sort}(a, n), n) \Rightarrow \text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, \text{sort}(a, n)[i] < v\}|.$$

Since we know the assumption on the left-hand side, we also know the right-hand side. Since $\text{perm}(a, \text{sort}(a, n), n)$, both a and $\text{sort}(a, n)$ contain exactly the same elements, and hence

$$\text{count}(\text{sort}(a, n), n, v) = |\{i \mid i \in \{1, \dots, n\}, a[i] < v\}|.$$

which is what we wanted to prove.

Implementation

Write input/output specifications as **assertions**:

function sort(a, n):

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $b \in \mathcal{A}$ s.t. $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

@ $a \in \mathcal{A}, n \in \mathcal{N}$

...

@ $\text{sorted}(b, n) \wedge \text{perm}(a, b, n)$

return b

function count(a, n, x):

Input: $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

Output: $|\{a[i] < x \mid i \in \{1, \dots, n\}\}|$

@ $a \in \mathcal{A}, n \in \mathcal{N}, x$ s.t. $\text{sorted}(a, n)$

...

@ $r = |\{i \mid i \in \{1, \dots, n\}, a[i] < x\}|$

return r

Functions: Summary

Each function should have an input/output specification.

function $f(v_1, \dots, v_n)$

Input: $I(v_1, \dots, v_n)$

Output: v' s.t. $O(v_1, \dots, v_n, v')$

that expresses the logical **formulas**

$$\forall v_1, \dots, v_n, v' . [I(v_1, \dots, v_n) \wedge v' = f(v_1, \dots, v_n)] \Rightarrow O(v_1, \dots, v_n, v')$$

$$\forall v_1, \dots, v_n . I(v_1, \dots, v_n) \Rightarrow O(v_1, \dots, v_n, f(v_1, \dots, v_n)).$$

This can be added as **assumptions** to the basic paths using the functions.

This adds them to the left-hand side of the corresponding verification conditions.

Here, we can fully **ignore** internal **code** of the function

Function implementation: I/O specification as first and last assertion

What Is a Function?

```
if(foo(1)==foo(1)) {  
    ...                               // send the user 100 Euro  
} else {  
    ...                               // format all hard disks  
}
```

In general, a C function is *not* a function! (in the mathematical sense)

A function always returns the **same result** for the **same argument**

Hence: no dependence on user input, global variables, file system etc.

User output is o.k., our method simply does not express anything about it

What Is a Function?

```
x= 1;  
z= foo(x);
```

```
if(x==1) {  
    ... // send the user 100 Euro  
} else {  
    ... // format all hard disks  
}
```

```
y[0]= 1;  
z= foo(y);
```

```
if(y[0]==1) {  
    ... // send the user 100 Euro  
} else {  
    ... // format all hard disks  
}
```

call by value versus call by reference

Unlike a C function, a function does **not modify** its **arguments**.

Recursive Function Calls: Example

$$r = \text{gcd}(x, y) :\Leftrightarrow r|x \wedge r|y \wedge \neg \exists r' . r'|x \wedge r'|y \wedge r' > r$$

function GCD(x, y)

Input: $x \in \mathcal{N}, y \in \mathcal{N}, x \geq y$

Output: $\text{gcd}(x, y)$

$$\forall x \in \mathcal{N}, y \in \mathcal{N} . [x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y)$$

Implementation:

function GCD(x, y)

@ $x, y \in \mathcal{N}, x \geq y$

if $y = 0$ **then**

$r \leftarrow x$

else

$r \leftarrow \text{GCD}(y, x \bmod y)$

@ $r = \text{gcd}(x, y)$

return r

Correctness proof?

Recursive Function Calls: Example

```
function GCD( $x, y$ )  
@  $x, y \in \mathcal{N}, x \geq y$   
if  $y = 0$  then  
     $r \leftarrow x$   
else  
     $r \leftarrow \text{GCD}(y, x \bmod y)$   
@  $r = \text{gcd}(x, y)$   
return  $r$ 
```

(demo)

Basic paths:

```
assume  $x, y \in \mathcal{N}, x \geq y$   
assume  $y = 0$   
 $r \leftarrow x$   
@  $r = \text{gcd}(x, y)$ 
```

```
assume  $\forall x \in \mathcal{N}, y \in \mathcal{N} .$   
     $[x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y)$   
assume  $x, y \in \mathcal{N}, x \geq y$   
assume  $y \neq 0$   
 $r \leftarrow \text{GCD}(y, x \bmod y)$   
@  $r = \text{gcd}(x, y)$ 
```

Proof of Second Verification Condition

We assume the left-hand side:

$$\forall x, y, r. [x \geq y \wedge r = \text{GCD}(x, y)] \Rightarrow r = \text{gcd}(x, y)$$

$$x \geq y$$

$$y \neq 0$$

$$r = \text{GCD}(y, x \bmod y)$$

and prove

$$r = \text{gcd}(x, y).$$

From the formula above, after choosing $x \leftarrow y, y \leftarrow x \bmod y$ we conclude

$$[y \geq x \bmod y \wedge r = \text{GCD}(y, x \bmod y)] \Rightarrow r = \text{gcd}(y, x \bmod y)$$

The part $y \geq x \bmod y$ is a mathematical fact. Moreover, we already know $r = \text{GCD}(y, x \bmod y)$. So we can conclude $r = \text{gcd}(y, x \bmod y)$.

Now it suffices to prove $\text{gcd}(y, x \bmod y) = \text{gcd}(x, y)$, which is a well-known mathematical fact.

Proving Correctness of Recursive Function Calls: Summary

Additional assumption:

logical formula corresponding to the I/O specification of the **function itself**

This is not a circular argument:

proof of base case of recursion does not depend on this formula

Again: the function must be side-effect free:

no input, no global variables

Specification of Procedures

no return value, but may modify arguments (call by reference)

Example:

procedure `reverse`(a, n)

Input: $a \in \mathcal{A}, n \in \mathcal{N}$

Output: $a^* \in \mathcal{A}$ s.t. $\forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$

Modifies a , hence we need a different name for the input and output value.

Further possibilities, e.g. a^{in}/a^{out} , $a\downarrow/a\uparrow$

Corresponding logical formula?

$$\forall a, a^*, n . \cancel{\text{reverse}(a, n)} \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

Separately list input and output argument:

$$\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

Here, *reverse* is a predicate!

Usage

assume $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$

assume $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$

reverse(a, n)

@ $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$

three vs. two arguments!? predicate vs. procedure

SSA:

assume $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$

assume $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$

assume $\text{reverse}(a, a_1, n)$

// *reverse is a predicate now*

@ $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$

Example: Proof of Verification Condition

Assumptions:

$$\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$$

$$\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$$

$$\text{reverse}(a, a_1, n)$$

$$\text{reverse}(a, a_1, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$$

$$\forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$$

$$i \in \{0, \dots, n-1\}$$

$$i \in \{0, \dots, n-1\} \Rightarrow a_1[i] = a[n-1-i]$$

$$a_1[i] = a[n-1-i]$$

$$a[n-1-i] \geq 0$$

To prove:

$$\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0 \quad a_1[i] \geq 0 \quad a[n-1-i] \geq 0$$

Example: Proof of Verification Condition

Assumptions:

- ▶ $\forall a, a^*, n . \text{reverse}(a, a^*, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$
- ▶ $\forall i \in \{0, \dots, n-1\} . a[i] \geq 0$
- ▶ $\text{reverse}(a, a_1, n)$

To prove:

- ▶ $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$

For the concrete call we know (substitutions $a \leftarrow a_1, n \leftarrow n$):

$$\text{reverse}(a, a_1, n) \Rightarrow \forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$$

Resulting new assumption: $\forall i \in \{0, \dots, n-1\} . a_1[i] = a[n-1-i]$

To prove $\forall i \in \{0, \dots, n-1\} . a_1[i] \geq 0$, let i be arbitrary, but fixed.

We prove $a_1[i] \geq 0$.

Due to the second assumption this means to prove $a[n-1-i] \geq 0$, which we know after substituting $i \leftarrow n-1-i$ in the second assumption above.

Implementation

procedure reverse(a, n)

Input: $a \in \mathcal{A}$, $n \in \mathcal{N}$

Output: $a^* \in \mathcal{A}$ s.t. $\forall i \in \{0, \dots, n-1\} . a^*[i] = a[n-1-i]$

procedure reverse(a, n)

@ $a \in \mathcal{A}$, $n \in \mathcal{N}$

$a^{in} \leftarrow a$

...

@ $\forall i \in \{0, \dots, n-1\} . a[i] = a^{in}[n-1-i]$

return

Procedures, More Parameters: Example

procedure *swap*(x, y)

Input:

Output: x^*, y^* s.t. $x^* = y, y^* = x$

Usage:

assume $\forall x, y, x^*, y^* . \text{swap}(x, x^*, y, y^*) \Rightarrow [x^* = y \wedge y^* = x]$

assume $x \geq 0 \wedge y \geq 1$

swap(x, y)

@ $x \geq 1 \wedge y \geq 0$

SSA:

assume $\forall x, y, x^*, y^* . \text{swap}(x, x^*, y, y^*) \Rightarrow [x^* = y \wedge y^* = x]$

assume $x \geq 0 \wedge y \geq 1$

assume *swap*(x, x_1, y, y_1)

@ $x_1 \geq 1 \wedge y_1 \geq 0$

Procedures: Summary

procedure $p(v_1, \dots, v_n)$

Input: $I(v_1, \dots, v_n)$

Output: $v_1^*, \dots, v_n^*$ s.t. $O(v_1, v_1^*, \dots, v_n, v_n^*)$

Assumption from specification:

$$\forall v_1, v_1^*, \dots, v_n, v_n^* .$$

$$[I(v_1, \dots, v_n) \wedge p(v_1, v_1^*, \dots, v_n, v_n^*)] \Rightarrow O(v_1, v_1^*, \dots, v_n, v_n^*)$$

A call $p(v_1, \dots, v_n)$ becomes **assume** $p(v_1, v_1^*, \dots, v_n, v_n^*)$ in SSA,
with p being a predicate symbol.

Further Examples

1. Recursive function calls (binary search)
2. Program development by assertion based stepwise refinement (sorting)

... in both cases array indices from 1 to n

Recursive Binary Search

function $\text{search}(a, l, u, x)$

Input: $a \in \mathcal{A}(\mathcal{N})$, $n \in \mathcal{N}$, $\text{sorted}(a, n)$, $l, u \in \{1, \dots, n\}$, $x \in \mathcal{N}$

Output: r , s.t.
$$\begin{aligned} r = 0 &\Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \wedge \\ r \neq 0 &\Rightarrow [l \leq r \leq u \wedge a[r] = x] \end{aligned}$$

As a formula:

$$\forall a, l, u, x, r . r = \text{search}(a, l, u, x) \Rightarrow \left[\begin{array}{l} r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \\ \quad \quad \quad \wedge \\ r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x] \end{array} \right]$$

Implementation

```
function search( $a, l, u, x$ )  
assume  $a \in \mathcal{A}(\mathcal{N})$ ,  $n \in \mathcal{N}$ , sorted( $a, n$ ),  $l, u \in \{1, \dots, n\}$ ,  $x \in \mathcal{N}$   
if  $u < l$  then  
     $r \leftarrow 0$   
else if  $a[(l + u)/2] = x$  then  
     $r \leftarrow (l + u)/2$   
else if  $a[(l + u)/2] < x$  then  
     $r \leftarrow \text{search}(a, (l + u)/2 + 1, u, x)$   
else  
     $r \leftarrow \text{search}(a, l, (l + u)/2 - 1, x)$   
@  $[r = 0 \Rightarrow [\neg \exists i. l \leq i \leq u \wedge a[i] = x]] \wedge [r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]]$   
return  $r$ 
```

In the correctness proof we ignore the input condition, it is preserved obviously.

Correctness proof of non-recursive branches: easy

First Recursive Branch

assume $\forall a, l, u, x, r . r = \text{search}(a, l, u, x) \Rightarrow$

$$\left[\begin{array}{l} r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x] \\ \wedge \\ r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x] \end{array} \right]$$

assume $a \in \mathcal{A}(\mathcal{N}), n \in \mathcal{N}, \text{sorted}(a, n), l, u \in \{1, \dots, n\}, x \in \mathcal{N}$

assume $a[(l + u)/2] < x$

$r \leftarrow \text{search}(a, (l + u)/2 + 1, u, x)$

@ $[r = 0 \Rightarrow [\neg \exists i . l \leq i \leq u \wedge a[i] = x]] \wedge [r \neq 0 \Rightarrow [l \leq r \leq u \wedge a[r] = x]]$

analyze this directly, or prove the corresponding verification condition

Sorting Algorithm

procedure sort(a, n)

Input: array a

Output: a^* s.t. $\text{perm}(a, a^*) \wedge \text{sorted}(a^*, n)$ where

$$\text{sorted}(a, n) :\Leftrightarrow \forall i \in \{1, \dots, n-1\} . a[i] \leq a[i+1]$$

$$\text{perm}(a, b, n) :\Leftrightarrow \exists c . \text{perm1n}(c, n) \wedge \forall i \in \{1, \dots, n\} . b[c[i]] = a[i]$$

$$\text{perm1n}(a, n) :\Leftrightarrow \forall i \in \{1, \dots, n\} \exists j \in \{1, \dots, n\} . a[j] = i$$

Sorting Algorithm: Auxiliary Procedure

procedure swap(a, n, i, j)

Input: array a , $i, j, n \in \mathcal{N}$, $1 \leq i \leq n$, $1 \leq j \leq n$

Output: a^* s.t.
$$\begin{cases} a^*[i] = a[j], \\ a^*[j] = a[i], \\ \forall k. [1 \leq k \leq n, k \neq i, k \neq j] \Rightarrow a^*[k] = a[k] \end{cases}$$

Sorting Algorithm: Stepwise Refinement

$\text{minfirst}(a, n, i) :\Leftrightarrow a[i] = \min\{a[k] \mid k \in \{i, \dots, n\}\}$

Example: $\text{minfirst}(\boxed{10 \mid 3 \mid 7 \mid 4 \mid 7}, 5, 2)$

procedure $\text{sort}(a, n)$

$a_{in} \leftarrow a$

for $i \leftarrow 1$ **to** n **do**

$\textcircled{\text{}} \text{perm}(a_{in}, a) \wedge \forall k \in \{1, \dots, i-1\} . \text{minfirst}(a, n, k)$

for $j \leftarrow n$ **down to** $i+1$ **do**

$\textcircled{\text{}} \text{perm}(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . \text{minfirst}(a, n, k)] \wedge \text{minfirst}(a, n, j)$

if $a[j-1] > a[j]$ **then** $\text{swap}(a, j-1, j)$

$\textcircled{\text{}} \text{perm}(a_{in}, a) \wedge [\forall k \in \{1, \dots, i-1\} . \text{minfirst}(a, n, k)] \wedge \text{minfirst}(a, n, j-1)$

$\textcircled{\text{}} \text{perm}(a_{in}, a) \wedge \forall k \in \{1, \dots, i\} . \text{minfirst}(a, n, k)$

$\textcircled{\text{}} \text{perm}(a_{in}, a) \wedge \text{sorted}(a, n)$

return

assume $\text{minfirst}(a, n, j)$

assume $a[j-1] > a[j]$

$\text{swap}(a, j-1, j)$

$\textcircled{\text{}} \text{minfirst}(a, n, j-1)$

Conclusion

For proving correctness of function and procedure calls

- ▶ we **assume** that the function/procedure returns for every input the **correct result**, and
- ▶ **prove correctness** of the call **under** this **assumption**.

The function/procedure specification allows us to **ignore** its **implementation**!

Stepwise refinement:

1. What?
2. How?